

AIN SHAMS UNIVERSITY

FACULTY OF ENGINEERING

Computer Vision Lab Assignment 01

OpenCV Image Processing

CSE480 / CSE482 Computer Vision Lab

Mechatronics Engineering and Automation Program

PREPARED BY

Student Name	Kerolus Hany Rabie Hana
Student ID	22P0134

1. Introduction

This report presents four image-processing tasks completed with Python, OpenCV, and NumPy. The tasks demonstrate selective colour isolation, weighted image blending, vignette filtering, and channel swapping. Each section states the objective, explains the processing steps, includes the code, and presents the resulting images.

2. Tasks and Workflow

The supplied apple, cameraman, zebra, and Mars rover images are loaded in OpenCV's BGR format. Each task writes its processed result to a results folder. The notebook uses `cv2_imshow` for inline display in Google Colab.

2.1 Common Setup Code

```
import cv2
import numpy as np
from pathlib import Path
from google.colab.patches import cv2_imshow

image_dir = Path(".")
output_dir = Path("results")
output_dir.mkdir(exist_ok=True)

def read_image(filename):
    image = cv2.imread(str(image_dir / filename))
    if image is None:
        raise FileNotFoundError(f"Could not read {filename}")
    return image
```

3. Task 1: Selective Colour Isolation

3.1 Objective

Keep red-dominant apple pixels in colour and convert the remaining image to grayscale.

3.2 Method and explanation

The image is converted to grayscale and split into blue, green, and red channels. A binary mask selects pixels where red is above 100 and exceeds both green and blue by more than 15. Bitwise operations combine the selected original pixels with the inverse-mask grayscale background.

3.3 Code

```
img = read_image("apple.png")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray_bgr = cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)
b, g, r = cv2.split(img)

mask1 = cv2.threshold(r, 100, 255, cv2.THRESH_BINARY)[1]
mask2 = cv2.compare(r.astype(np.int16), g.astype(np.int16) + 15, cv2.CMP_GT)
mask3 = cv2.compare(r.astype(np.int16), b.astype(np.int16) + 15, cv2.CMP_GT)
mask = cv2.bitwise_and(cv2.bitwise_and(mask1, mask2), mask3)

red_apple = cv2.bitwise_and(img, img, mask=mask)
gray_background = cv2.bitwise_and(
    gray_bgr, gray_bgr, mask=cv2.bitwise_not(mask)
)
result1 = cv2.add(red_apple, gray_background)

cv2.imwrite("results/task1_grayscale.png", gray_bgr)
cv2.imwrite("results/task1_mask.png", mask)
cv2.imwrite("results/task1_result.png", result1)
cv2_imshow(gray_bgr)
cv2_imshow(mask)
cv2_imshow(img)
cv2_imshow(result1)
```

3.4 Output

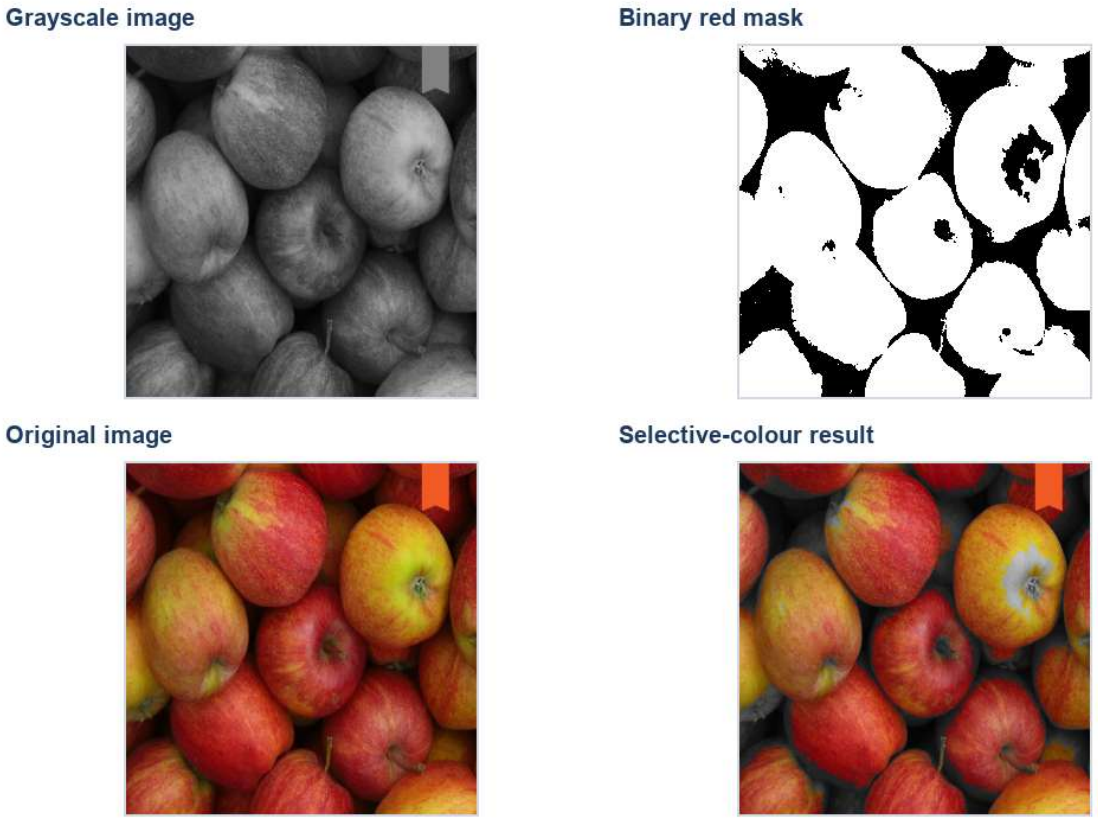


Figure 1. *Grayscale image, red-dominance mask, original image, and selective-colour composite.*

4. Task 2: Creating a Ghostly Overlay

4.1 Objective

Blend the cameraman and zebra images into a translucent composite.

4.2 Method and explanation

The zebra image is resized to match the cameraman dimensions. `cv2.addWeighted()` combines both images with equal 0.5 weights and zero brightness offset, so each source contributes equally.

4.3 Code

```
img1 = read_image("cameraman.jpg")
img2 = read_image("zebra.png")
img2 = cv2.resize(img2, (img1.shape[1], img1.shape[0]))

result2 = cv2.addWeighted(img1, 0.5, img2, 0.5, 0)
cv2.imwrite("results/task2_ghost.png", result2)

cv2_imshow(img1)
cv2_imshow(img2)
cv2_imshow(result2)
```

4.4 Output



Figure 2. *Source images and their equal-weight overlay.*

5. Task 3: Artistic Vignette Filtering

5.1 Objective

Darken the outer edges gradually while preserving a brighter central region.

5.2 Method and explanation

A black mask is created and a filled white circle is drawn at the image centre. Gaussian blur softens the circular boundary. The mask is normalized to 0–1 and multiplied into the image, reducing brightness toward the edges.

5.3 Code

```
img = read_image("marsrover.png")
height, width = img.shape[:2]
mask = np.zeros((height, width), dtype=np.uint8)
center = (width // 2, height // 2)
radius = min(height, width) // 2
cv2.circle(mask, center, radius, 255, -1)
mask = cv2.GaussianBlur(mask, (101, 101), 50)

mask_float = mask.astype(np.float32) / 255.0
result3 = (img.astype(np.float32) * mask_float[:, :, None])
result3 = result3.clip(0, 255).astype(np.uint8)
cv2.imwrite("results/task3_mask.png", mask)
cv2.imwrite("results/task3_vignette.png", result3)
cv2_imshow(mask)
cv2_imshow(result3)
```

5.4 Output



Figure 3. *Soft vignette mask and Mars rover image after application.*

6. Task 4: Channel Swapping

6.1 Objective

Produce a surreal palette by merging the zebra image channels in a different order.

6.2 Method and explanation

OpenCV stores colour images in BGR order. `cv2.split()` separates the channels and `cv2.merge()` recombines them as red, blue, green. This changes the represented colours while leaving pixel locations unchanged.

6.3 Code

```
img = read_image("zebra.png")
b, g, r = cv2.split(img)
result4 = cv2.merge([r, b, g])
cv2.imwrite("results/task4_channel_swap.png", result4)

cv2_imshow(img)
cv2_imshow(result4)
```

6.4 Output



Figure 4. *Original zebra and the channel-swapped result.*

7. Conclusion

The four exercises show how masks, channel values, weighted arithmetic, and Gaussian blur can produce distinct image effects. The final images demonstrate selective colour, a translucent overlay, a soft vignette, and a surreal channel rearrangement.

Links

[1] https://github.com/KerolusHany/Computer-Vision-Course/tree/main/LAB%2001